gasmodel: An R Package for Generalized Autoregressive Score Models

by Vladimír Holý

Abstract Generalized autoregressive score (GAS) models are a class of observation-driven time series models that employ the score to dynamically update time-varying parameters of the underlying probability distribution. GAS models have been extensively studied and numerous variants have been proposed in the literature to accommodate diverse data types and probability distributions. This paper introduces the gasmodel package, which has been designed to facilitate the estimation, forecasting, and simulation of a wide range of GAS models. The package provides a rich selection of distributions, offers flexible options for specifying dynamics, and allows users to incorporate exogenous variables. Model estimation utilizes the maximum likelihood method.

1 Introduction

The generalized autoregressive score (GAS) models, introduced by Creal et al. (2013) and Harvey (2013), have emerged as a valuable and contemporary framework for time series modeling. These models, also referred to as dynamic conditional score (DCS) models or score-driven models, offer flexibility by accommodating various underlying probability distributions and time-varying parameters. GAS models are observation-driven, effectively capturing the dynamic behavior of time-varying parameters through the autoregressive term and the score, i.e., the gradient of the log-likelihood function. The GAS framework enables the formulation of a wide range of dynamic models suitable for many commonly used univariate and multivariate data types.

There are several packages and code available in R that handle GAS models. One notable package is **GAS** developed by Ardia et al. (2018) and Ardia et al. (2019), which provides functionality for both univariate and multivariate GAS models. The current version of the **GAS** package, 0.3.4, supports 16 distributions. However, the model specification in the **GAS** package is somewhat limited, only allowing for basic dynamics without the inclusion of exogenous variables. Additionally, this package lacks distributions for certain more specialized data types, such as circular, compositional, and ranking data. The package thus supports only a limited selection of GAS models found in the literature. For a more detailed comparison of the **gasmodel** and **GAS** packages, see the `comparison_gas` vignette from the **gasmodel** package. Another relevant R package is **betategarch** by Sucarrat (2013), which deals specifically with the Beta-Skew-t-EGARCH model, a GAS model for time-varying volatility based on the Student's t-distribution. In Python, the PyFlux library by Taylor (2018) deals with time series analysis and features various GAS models including the Beta-Skew-t-EGARCH model, standard GAS models, GAS random walk models, GAS pairwise comparison models, and GAS regression models. In Julia, the ScoreDrivenModels.j package by Bodin et al. (2020) provides a framework for standard GAS models. The Time Series Lab program by Lit et al. (2021) is a stand-alone GUI application designed to model and forecast time series, including standard GAS models, GAS pairwise comparison models, and GAS regression models. Additional R, Python, MATLAB, and Ox code for some specific GAS models, often associated with individual research papers, can be found on the `www.gasmodel.com` website.

In this paper, we present the **gasmodel** package, which is designed to provide comprehensive functionality that encompasses a wide range of GAS models documented in the existing literature. It offers versatile model specification and core features available for the entire spectrum of implemented distributions. The current version of the **gasmodel** package, 0.6.2, offers a selection of 36 distributions, catering to various univariate and multivariate data types such as binary, categorical, ranking, count, integer, circular, interval, compositional, duration, and real data types. A comprehensive list of these distributions is provided in Table 1. For details on each distribution, see the `distributions` vignette from the **gasmodel** package. Model specification within the package allows for flexible customization, enabling users to incorporate different parametrizations, exogenous variables, joint and separate modeling of exogenous variables and dynamics, higher score and autoregressive orders, custom and unconditional initial values of time-varying parameters, fixed and bounded values of coefficients, and missing values. Model estimation is performed by the maximum likelihood method. Standard errors of coefficients are estimated using the empirical Hessian matrix. Furthermore, the package offers a range of functionalities including forecasting, simulation, bootstrapping, and assessment of parameter uncertainty. Comprehensive documentation is provided with the package, offering details on each distribution and its corresponding parametrizations.

The **gasmodel** package is accessible on CRAN at `cran.r-project.org/package=gasmodel`. Addi-

Table 1: List of available distributions and their parametrizations. First parametrization is the default.

Label	Distribution	Dim.	Data Type	Parametrizations
`alaplace`	Asymmetric Laplace	Uni.	Real	`meanscale`
`bernoulli`	Bernoulli	Uni.	Binary	`prob`
`beta`	Beta	Uni.	Interval	`conc,meansize,meanvar`
`bisa`	Birnbaum-Saunders	Uni.	Duration	`scale`
`burr`	Burr	Uni.	Duration	`scale`
`cat`	Categorical	Multi.	Categorical	`worth`
`dirichlet`	Dirichlet	Multi.	Compositional	`conc`
`dpois`	Double Poisson	Uni.	Count	`mean`
`explog`	Exponential-Logarithmic	Uni.	Duration	`rate`
`exp`	Exponential	Uni.	Duration	`scale,rate`
`fisk`	Fisk	Uni.	Duration	`scale`
`gamma`	Gamma	Uni.	Duration	`scale,rate`
`ged`	Generalized Error	Uni.	Real	`meanscale`
`gengamma`	Generalized Gamma	Uni.	Duration	`scale,rate`
`geom`	Geometric	Uni.	Count	`mean,prob`
`kuma`	Kumaraswamy	Uni.	Interval	`conc`
`laplace`	Laplace	Uni.	Real	`meanscale`
`logistic`	Logistic	Uni.	Real	`meanscale`
`logitnorm`	Logit-Normal	Uni.	Interval	`logitmeanvar`
`lognorm`	Log-Normal	Uni.	Duration	`logmeanvar`
`lomax`	Lomax	Uni.	Duration	`scale`
`mvnorm`	Multivariate Normal	Multi.	Real	`meanvar`
`mvt`	Multivariate Student's t	Multi.	Real	`meanvar`
`negbin`	Negative Binomial	Uni.	Count	`nb2,prob`
`norm`	Normal	Uni.	Real	`meanvar`
`pluce`	Plackett-Luce	Multi.	Ranking	`worth`
`pois`	Poisson	Uni.	Count	`mean`
`rayleigh`	Rayleigh	Uni.	Duration	`scale`
`skellam`	Skellam	Uni.	Integer	`meanvar,diff,meandisp`
`t`	Student's t	Uni.	Real	`meanvar`
`vonmises`	von Mises	Uni.	Circular	`meanconc`
`weibull`	Weibull	Uni.	Duration	`scale,rate`
`zigeom`	Zero-Inflated Geometric	Uni.	Count	`mean`
`zinegbin`	Zero-Inflated Negative Binomial	Uni.	Count	`nb2`
`zipois`	Zero-Inflated Poisson	Uni.	Count	`mean`
`ziskellam`	Zero-Inflated Skellam	Uni.	Integer	`meanvar,diff,meandisp`

tionally, users can find the development version of the package on GitHub at `github.com/vladimirholy/gasmodel`, providing them with the opportunity to report any bugs or issues they encounter.

The rest of the paper is as follows. In Section 2, we outline the key characteristics of GAS models. In Section 3, we present an overview of the **gasmodel** package. In Section 4, we present a case study demonstrating the practical application of the package. We conclude the paper in Section 5.

2 Generalized autoregressive score models

2.1 Background

The concept of utilizing the score as a driving mechanism for dynamics in time series was independently developed at both Vrije Universiteit Amsterdam and the University of Cambridge. At Vrije Universiteit Amsterdam, researchers established a comprehensive general methodology that encompasses various models driven by the score, known as the generalized autoregressive score (GAS) models. The initial findings were presented in the working paper Creal et al. (2008), which was subsequently published as Creal et al. (2013). At the University of Cambridge, the initial focus was on a specific model that employed the Student's t-distribution with dynamic volatility, named Beta-t-(E)GARCH. This approach was introduced in a working paper Harvey and Chakravarty (2008). The book by Harvey (2013) explores a variety of dynamic location and scale models driven by the score, referring to them as dynamic conditional score (DCS) models. Both Creal et al. (2013) and Harvey (2013) are widely recognized as seminal contributions to the literature on GAS models. More

recently, in order to reconcile different terminologies used in the literature, the term "score-driven models" has also emerged as a synonymous label.

The Scopus database reports 799 articles containing the phrase "generalized autoregressive score" or "dynamic conditional score", as of August 17, 2025. The website www.gasmodel.com lists 438 articles, working papers, and books on GAS models, as of August 11, 2025.

2.2 Basic notation

The goal is to model time series y_t, $t = 1, \ldots, T$, which can be univariate or multivariate, continuous or discrete. Let f_t denote the vector of time-varying parameters and g the vector of static parameters. Let $p(y_t|f_t, g)$ denote the density function in the case of a continuous variable, or the probability mass function in the case of a discrete variable.

Constructing a model involves two main components: selecting an appropriate distribution and specifying the dynamics of its time-varying parameters.

2.3 Score as the key ingredient

In GAS models, the key ingredient driving the dynamics of the parameter vector f_t is the score, i.e., the gradient of the log-likelihood function,

$$\nabla(y_t, f_t) = \frac{\partial \ln p(y_t|f_t, g)}{\partial f_t}. \tag{1}$$

The score has zero expected value and its variance is known as the Fisher information,

$$\mathcal{I}(f_t) = \mathrm{E}\left[\left(\frac{\partial \ln p(y_t|f_t, g)}{\partial f_t}\right)^2 \middle| f_t, g\right]. \tag{2}$$

The score quantifies the discrepancy between the fitted distribution, determined by f_t, and a particular observation y_t. As such, it can be employed as a correction term following the realization of an observation. When the score is positive, it suggests that the parameter of interest should be increased to better accommodate the observed data. Conversely, when the score is negative, decreasing the parameter would help in aligning the distribution with the observation. When the score is zero, it indicates that the current parameter value represents the optimal fit for the specific observation at hand.

An advantage of the score is that it takes into account the shape of the distribution. To illustrate this point, Creal et al. (2013) consider two GARCH models: one based on the normal distribution and another based on the Student's t-distribution. Now, imagine an extreme observation occurs. Due to its heavier tails, the Student's t-distribution assigns a higher probability to such extreme observations compared to the normal distribution. Crucially, this distinction is also mirrored in the score. Specifically, when assuming the normal distribution, the score for the extreme observation will have a significantly higher absolute value compared to when assuming the Student's t-distribution. The dynamics can thus reflect the shape of the distribution.

The simple difference between expectation and realization, commonly used as a correction term in various time series models, may not always be effective for distributions with specific support. Harvey et al. (2024) highlight this limitation in the context of circular time series. To illustrate this, let us suppose the expected value of an angle is 0.01 radians, but the actual observation turns out to be 6.27 radians. Although the numerical difference between these values is substantial, their corresponding angles are very similar as 0 radians and 2π radians represent the exact same angle. This discrepancy highlights the inadequacy of using a simple difference metric. On the other hand, the score respects the circular nature of the data. For instance, when working with the von Mises distribution characterized by a time-varying location parameter μ_t and a static concentration parameter ν, the score for μ_t is equal to $\nu \sin(y_t - \mu_t)$. By employing the sine function, the score accounts for the circularity of the data and ensures that the angular differences are appropriately considered during the analysis.

2.4 Dynamics of time varying parameters

In GAS models, time-varying parameters f_t follow the recursion

$$f_t = \omega + \sum_{j=1}^{P} \alpha_j S(f_{t-j}) \nabla(y_{t-j}, f_{t-j}) + \sum_{k=1}^{Q} \varphi_k f_{t-k}, \tag{3}$$

where ω is the intercept, α_j are the score parameters, φ_k are the autoregressive parameters, and $S(f_t)$ is a scaling function for the score. In the case of a single time-varying parameter, all these quantities are scalar. In the case of multiple time-varying parameters, ω and $\nabla(y_{t-j}, f_{t-j})$ are vectors, α_j and φ_k are diagonal matrices, and $S(f_t)$ is a square matrix. In the majority of empirical studies, it is common practice to set the score order P and the autoregressive order Q to 1. Furthermore, one of three scaling functions is typically chosen: the unit function, the inverse of the Fisher information, or the square root of the inverse of the Fisher information. When the latter is used, the scaled score has unit variance. However, the choice of the scaling function is not always a straightforward task and is closely tied to the underlying distribution.

The dynamics of the model can be expanded to incorporate exogenous variables as

$$f_t = \omega + \sum_{i=1}^{M} \beta_i x_{ti} + \sum_{j=1}^{P} \alpha_j S(f_{t-j}) \nabla(y_{t-j}, f_{t-j}) + \sum_{k=1}^{Q} \varphi_k f_{t-k}, \tag{4}$$

where β_i are the regression parameters associated with the exogenous variables x_{ti}. Alternatively, a different model can be obtained by defining the recursion in the fashion of regression models with dynamic errors as

$$f_t = \omega + \sum_{i=1}^{M} \beta_i x_{ti} + e_t, \quad e_t = \sum_{j=1}^{P} \alpha_j S(f_{t-j}) \nabla(y_{t-j}, f_{t-j}) + \sum_{k=1}^{Q} \varphi_k e_{t-k}. \tag{5}$$

The key distinction between the two models lies in the impact of exogenous variables on f_t. Specifically, in the former model formulation, exogenous variables influence all future parameters through both the autoregressive term and the score term. In the latter model formulation, they affect future parameters only through the score term. When no exogenous variables are included, the two specifications are equivalent, although with differently parameterized intercepts. When numerically optimizing parameter values, the latter model exhibits faster convergence, thanks to the dissociation of ω from φ_k. In the context of ARIMA models, the differences between the two model specifications are discussed by Hyndman (2010). In the context of a GAS model for ranking data, Holý (2025) advocates the use of a regression model with dynamic errors in an application to ice hockey results.

Other model specifications can be obtained by imposing various restrictions on ω, β_i, α_j, or φ_k. In addition, it is possible to have different orders P and Q for individual parameters when multiple parameters are time-varying. Furthermore, the set of exogenous variables can also vary for different parameters.

The recursive nature of f_t necessitates the initialization of the first few elements $f_1, \ldots, f_{\max\{P,Q\}}$. A sensible approach is to set them to the long-term value,

$$\bar{f} = \begin{cases} \left(1 - \sum_{k=1}^{Q} \varphi_k\right)^{-1} \left(\omega + \sum_{i=1}^{M} \beta_i \frac{1}{N} \sum_{t=1}^{N} x_{ti}\right) & \text{in model (4),} \\ \omega + \sum_{i=1}^{M} \beta_i \frac{1}{N} \sum_{t=1}^{N} x_{ti} & \text{in model (5).} \end{cases} \tag{6}$$

2.5 Maximum likelihood estimation

GAS models can be straightforwardly estimated by the maximum likelihood method. Let $\theta = (\omega, \beta_1, \ldots, \beta_M, \alpha_1, \ldots, \alpha_P, \varphi_1, \ldots, \varphi_Q, g)'$ denote the vector of all parameters to be estimated. The estimate $\hat{\theta}$ is then obtained by maximizing the full log-likelihood as

$$\hat{\theta} = \arg\max_{\theta} \sum_{t=1}^{T} \ln p(y_t | f_t, g). \tag{7}$$

Alternatively, the conditional log-likelihood can be maximized, which excludes the initial $\max\{P, Q\}$ terms. The maximization of the log-likelihood function can be accomplished using various general-purpose algorithms designed for solving nonlinear optimization problems.

The standard errors of the estimated parameters can be obtained using the standard maximum likelihood asymptotics. Under appropriate regularity conditions, the maximum likelihood estimator $\hat{\theta}$ is consistent and asymptotically normal. Specifically, it satisfies:

$$\sqrt{T}(\hat{\theta} - \theta_0) \xrightarrow{\mathrm{d}} \mathrm{N}(0, -\mathrm{E}[H(\theta_0)]^{-1}), \tag{8}$$

where θ_0 represents the true parameter values and $H(\theta)$ denotes the Hessian of the log-likelihood, defined as

$$H(\theta) = \frac{\partial^2 \ln p(y_t | f_t, g)}{\partial \theta \partial \theta'}. \tag{9}$$

In finite samples, the expected value of the Hessian can be approximated by the empirical Hessian of the log-likelihood evaluated at the estimated parameter values $\hat{\theta}$. This empirical Hessian provides an estimate of the curvature of the log-likelihood function and serves as a practical substitute for the true expected value of the Hessian when finite-sample inference is required.

The conditions for the consistency and asymptotic normality of the estimator depend on the specific distributional assumptions and dynamics of the model and need to be verified on a case-by-case basis. Each distribution may have its own specific characteristics and requirements for maximum likelihood estimation. For the general asymptotic theory regarding GAS models and maximum likelihood estimation, see Blasques et al. (2014), Blasques et al. (2018), and Blasques et al. (2022).

2.6 Theoretical and empirical properties

The use of the score for updating time-varying parameters is optimal in an information theoretic sense. For an investigation of the optimality properties of GAS models, see Blasques et al. (2015) and Blasques et al. (2021).

Generally, the GAS models perform quite well when compared to alternatives, including parameter-driven models. For a comparison of the GAS models to alternative models, see Koopman et al. (2016) and Blazsek and Licht (2020).

2.7 Notable models

The GAS class includes many well-known econometric models, such as the generalized autoregressive conditional heteroskedasticity (GARCH) model of Bollerslev (1986) based on the normal distribution, the autoregressive conditional duration (ACD) model of Engle and Russell (1998) based on the exponential distribution, and the count model of Davis et al. (2003) based on the Poisson distribution.

More recently, a variety of novel score-driven models has been proposed, such as the Beta-t-(E)GARCH model of Harvey and Chakravarty (2008), a multivariate Student's t volatility model of Creal et al. (2011), a Dirichlet model of Calvori et al. (2013), the GRAS copula model of De Lira Salvatierra and Patton (2015), the realized Wishart-GARCH model of Hansen et al. (2016), a bimodal Birnbaum–Saunders model of Fonseca and Cribari-Neto (2018), a Skellam model of Koopman et al. (2018), a circular model of Harvey et al. (2024), a Bradley–Terry model of Gorgi et al. (2019), a bivariate Poisson model of Koopman and Lit (2019), a censoring model of Harvey and Ito (2020), a double Poisson mixture model of Holý and Tomanová (2022), a ranking model of Holý and Zouhar (2022), a Tobit model of Harvey and Liao (2023), and a zero-inflated negative binomial model of Blasques et al. (2024).

For an overview of various GAS models, see Artemova et al. (2022) and Harvey (2022).

3 Features of the package

3.1 Model specification and estimation

The heart of the **gasmodel** package is the `gas()` function, which serves as a powerful tool for estimating both univariate and multivariate GAS models. This function offers extensive flexibility with its wide range of arguments:

```
gas(y, x = NULL, distr, param = NULL, scaling = "unit", regress = "joint",
  p = 1L, q = 1L, par_static = NULL, par_link = NULL, par_init = NULL,
  lik_skip = 0L, coef_fix_value = NULL, coef_fix_other = NULL,
  coef_fix_special = NULL, coef_bound_lower = NULL, coef_bound_upper = NULL,
  coef_start = NULL, optim_function = wrapper_optim_nloptr, optim_arguments =
  list(opts = list(algorithm = "NLOPT_LN_NELDERMEAD", xtol_rel = 0, maxeval =
  1e+06)), hessian_function = wrapper_hessian_stats, hessian_arguments = list(),
  print_progress = FALSE)
```

However, at its core, it only requires two essential inputs: a time series `y` and a distribution `distr`. All other arguments come with default values, ensuring that the function can be readily used even with minimal specifications.

A time series `y` can be represented as either a vector of length T or a $T \times 1$ matrix in the case of univariate series. In the multivariate case, it should be a $T \times N$ matrix, where N denotes the dimension of the series.

Additionally, there is an option to include exogenous variables `x`. When incorporating a single variable that is common for all time-varying parameters, a numeric vector of length T can be provided. For multiple variables that are common for all time-varying parameters, a $T \times M$ numeric matrix can be used. In cases where there are individual variables for each time-varying parameter, a list of numeric vectors or matrices following the aforementioned formats can be utilized. To control whether the variables are included in the dynamics equation together, as in (4), the argument `regress` can be set to the value `"joint"`. Alternatively, if separate equations for dynamics and regression are preferred, as in (5), `regress` can be set to the value `"sep"`. The default behavior is `regress = "joint"`, which is the model users might expect when incorporating exogenous variables. However, in general, we recommend using `regress = "sep"` to improve the numerical performance of the optimizer and to provide more straightforward interpretability of the model.

The selection of the distribution in the `gas()` function is determined by the `distr` argument. Some distributions have multiple parametrizations available, which can be specified using the `param` argument. It is important to note that certain parameters may have restrictions imposed on them, and these restrictions should be considered in the model dynamics. However, it may not always be possible to satisfy these restrictions, or it may require additional constraints on the coefficients controlling the dynamics. To handle parameter restrictions, it is generally recommended to use a link function that transforms the parameters into unrestricted real numbers. By default, the logarithmic function is applied to time-varying parameters in the interval $(0, \infty)$, while the logistic function is used for time-varying parameters in the interval $(0, 1)$. The static parameters are unaffected. This behavior can be modified by the `par_link` argument, which takes the form of a logical vector. The `TRUE` values indicate that the logarithmic/logistic link is applied to the corresponding parameters. The list of available distributions and their parametrizations can be obtained using the `distr()` function. Alternatively, Table 1 provides the relevant information.

The determination of time-varying and static parameters is guided by the `par_static` argument, which takes the form of a logical vector. The `TRUE` values indicate static parameters. By default, the first parameter of the distribution is considered time-varying, while the remaining parameters are treated as static. The score order P and the autoregressive order Q are selected by the `p` and `q` arguments respectively. These arguments can take either a single non-negative integer or a vector of non-negative integers when different orders are required for different parameters.

The choice of scaling function for the score is determined by the `scaling` argument. The supported scaling options include the unit scaling (`scaling = "unit"`), the scaling based on the inverse of the Fisher information matrix (`scaling = "fisher_inv"`), and the scaling based on the inverse square root of the Fisher information matrix (`scaling = "fisher_inv_sqrt"`). The latter two scalings utilize the Fisher information for the time-varying parameters exclusively. If the preference is to use the full Fisher information matrix, which includes both time-varying and static parameters, the `"full_fisher_inv"` or `"full_fisher_inv_sqrt"` scaling options can be selected. For the individual Fisher information associated with each parameter, the `"diag_fisher_inv"` and `"diag_fisher_inv_sqrt"` scaling options are available. It should be noted that when the parametrization is orthogonal (see `distr()`), there are no differences among these scaling variants.

The first $\max\{P, Q\}$ initial values of the time-varying parameters are by default set to their long-term values (6). It is also possible to assign specific values to the initial parameters using the `par_init` argument. During the maximization of the log-likelihood, the initial values can be included, resulting in the computation of the full likelihood, which is the default option. Alternatively, the initial values can be omitted, leading to the computation of the conditional likelihood by specifying `lik_skip = NULL`. To exclude a specified number of first few values from the likelihood calculation, a non-negative integer can be provided to `lik_skip`.

Restrictions on estimated coefficients can be enforced using several arguments. The `coef_fix_value` argument allows coefficients to be fixed at specific values, using a numeric vector where `NA` values indicate coefficients that are not fixed. To set coefficients as linear combinations of other coefficients, the `coef_fix_other` argument can be used. It requires a square matrix with multiples of the estimated coefficients, which are added to the fixed coefficients. A coefficient given by row is fixed on a coefficient given by column. By this logic, all rows corresponding to the estimated coefficients should contain only `NA` values. All columns corresponding to the fixed coefficients should also contain only `NA` values. For convenience, common coefficient structures can be specified by name using the `coef_fix_special` argument. Examples include `panel_structure`, `zero_sum_intercept`, and `random_walk`. To impose lower and upper bounds on coefficients, the `coef_bound_lower` and `coef_bound_upper` arguments can be utilized, respectively.

The `coef_start` argument allows for the specification of the starting values of coefficients used in the optimization process. If no values are provided, the starting values are automatically selected from a small grid of values. To define the optimization function, the `optim_function` argument is used. The function should be formatted according to the required specifications. Two wrapper functions are avail-

able for convenience: `wrapper_optim_stats()`, which utilizes the `optim()` function from the `stats` package, and `wrapper_optim_nloptr()`, which utilizes the `nloptr()` function from the **nloptr** package (Ypma and Johnson, 2020). Additional arguments can be passed to the optimization function as a list using the `optim_arguments` argument. Similarly, the Hessian matrix can be computed using the function specified in the `hessian_function` argument. Three wrappers are available: `wrapper_hessian_stats` for the `optimHess()` function from the `stats` package, `wrapper_hessian_pracma` for the `hessian()` function from the **pracma** package (Borchers, 2023), and `wrapper_hessian_numderiv` for the `hessian()` function from the **numDeriv** package (Gilbert and Varadhan, 2022). Additional arguments for the Hessian function can be passed as a list using the `hessian_arguments` argument. If desired, a detailed computation report can be continuously printed by setting the `print_progress` argument to `TRUE`.

The function returns a list of S3 class `gas`. This list consists of five components: `data`, `model`, `control`, `solution`, and `fit`, each of which is also a list. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `control` component contains the settings that control the optimization and Hessian computation. The `solution` component contains the raw results of the optimization and Hessian computation. Lastly, and most importantly, the `fit` component contains comprehensive estimation results. When an object of the `gas` class is printed, it provides a concise summary similar to the `summary.lm()` function from the `stats` package. Various generic functions can be applied to `gas` objects, including `summary()`, `plot()`, `coef()`, `vcov()`, `fitted()`, `residuals()`, `logLik()`, `AIC()`, `BIC()`, and `confint()`.

3.2 Forecasting

Forecasting of GAS models is performed using the `gas_forecast()` function. This function offers two forecasting methods. The `mean_path` method filters the time-varying parameters based on zero score and then generates the mean of the time series. The `simulated_paths` method repeatedly simulates time series, simultaneously filters time-varying parameters, and then estimates mean, standard deviation, and quantiles. See Blasques et al. (2016b) for more details on this method.

To use the `gas_forecast()` function, an estimated GAS model is required. Typically, the output of the `gas()` function (a `gas` object) can be supplied via the `gas_object` argument:

```
gas_forecast(gas_object, method = "mean_path", t_ahead = 1L, x_ahead = NULL,
  rep_ahead = 1000L, quant = c(0.025, 0.975))
```

Alternatively, multiple arguments including the data, model specification, and estimated coefficients can be manually specified:

```
gas_forecast(method = "mean_path", t_ahead = 1L, x_ahead = NULL,
  rep_ahead = 1000L, quant = c(0.025, 0.975), y, x = NULL, distr, param = NULL,
  scaling = "unit", regress = "joint", p = 1L, q = 1L, par_static = NULL,
  par_link = NULL, par_init = NULL, coef_est = NULL)
```

The forecasting method is determined by the `method` argument. The number of observations to forecast can be specified using the `t_ahead` argument. If exogenous variables are utilized, their values must be provided for the forecasted period using the `x_ahead` argument. For the `simulated_paths` method, the number of simulations can be controlled using the `rep_ahead` argument, and the desired quantiles can be specified using the `quant` argument.

The function returns a list of S3 class `gas_forecast` with three components: `data`, `model`, `forecast`. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `forecast` component contains the mean of the forecasted observations, along with standard deviations and quantiles if the `simulated_paths` method is used. Available generic functions are `summary()` and `plot()`.

3.3 Simulation

Basic simulation of GAS models is handled by the `gas_simulate()` function.

The `gas_simulate()` function requires supplying the coefficients using the `coef_est` argument and specifying the model using arguments `distr`, `param`, `scaling`, `regress`, `p`, `q`, `par_static`, `par_link`, `par_init`, and, in the case of multivariate models, the dimension `n`:

```
gas_simulate(t_sim = 1L, x_sim = NULL, distr, param = NULL, scaling = "unit",
  regress = "joint", n = NULL, p = 1L, q = 1L, par_static = NULL,
  par_link = NULL, par_init = NULL, coef_est = NULL)
```

Alternatively, only a gas object containing a model estimated by the gas() function can be provided using the gas_object argument:

```
gas_simulate(gas_object, t_sim = 1L, x_sim = NULL)
```

The number of observations to simulate can be specified using the t_sim argument. If exogenous variables are utilized, their values must be provided for the simulation sample using the x_sim argument.

The function returns a list of S3 class gas_simulate with three components: data, model, simulation. The data component contains the exogenous variables, if supplied. The model component contains the specification of the model structure and size. The simulation component contains the simulated time series, time-varying parameters, and scores. Available generic functions are summary() and plot().

3.4 Bootstrapping

To compute standard deviations and confidence intervals of the estimated coefficients in GAS models, the package provides the gas_bootstrap() function. This function employs the bootstrapping technique to estimate the uncertainty associated with the coefficients. The parametric method involves repeatedly simulating time series using the parametric model and re-estimating the coefficients based on the simulated data. The simple_block, moving_block, and stationary_block methods execute the circular block bootstrap with fixed non-overlapping blocks, fixed overlapping blocks, and randomly sized overlapping blocks, respectively.

The gas_bootstrap() function requires an estimated GAS model as input. The best way is to simply supply a gas object to the gas_object argument:

```
gas_bootstrap(gas_object, method = "parametric", rep_boot = 1000L,
  block_length = NULL, quant = c(0.025, 0.975), optim_function =
  wrapper_optim_nloptr, optim_arguments = list(opts = list(algorithm =
  "NLOPT_LN_NELDERMEAD", xtol_rel = 0, maxeval = 1e+04)), parallel_function =
  NULL, parallel_arguments = list())
```

Alternatively, the individual arguments including the data, model specification, and estimated coefficients can be provided:

```
gas_bootstrap(method = "parametric", rep_boot = 1000L, block_length = NULL,
  quant = c(0.025, 0.975), y, x = NULL, distr, param = NULL, scaling = "unit",
  regress = "joint", p = 1L, q = 1L, par_static = NULL, par_link = NULL,
  par_init = NULL, lik_skip = 0L, coef_fix_value = NULL, coef_fix_other = NULL,
  coef_fix_special = NULL, coef_bound_lower = NULL, coef_bound_upper = NULL,
  coef_est = NULL, optim_function = wrapper_optim_nloptr, optim_arguments =
  list(opts = list(algorithm = "NLOPT_LN_NELDERMEAD", xtol_rel = 0, maxeval =
  1e+04)), parallel_function = NULL, parallel_arguments = list())
```

The bootstrapping method is determined by the method argument. The number of bootstrap samples is specified by the rep_boot argument. For the simple_block and moving_block methods, the fixed size of blocks must be specified by the block_length argument. For the stationary_block method, the mean size of blocks must be specified by the block_length argument. The desired quantiles can be specified using the quant argument. As bootstrapping can be computationally very demanding, parallelization is achievable by employing the parallel_function argument, which expects a function similar to lapply(), allowing the application of a function over a list. Two wrapper functions are available for convenience: wrapper_parallel_multicore(), which utilizes the multicore parallelization functionality from the parallel package, and wrapper_parallel_snow(), which utilizes the snow parallelization functionality from the parallel package. Additional arguments can be passed to the parallelization function as a list using the parallel_arguments argument. If parallel_function is set to NULL, no parallelization is employed and lapply() is used.

The function returns a list of S3 class gas_bootstrap with three components: data, model, bootstrap. The data component contains the supplied time series and exogenous variables. The model component contains the specification of the model structure and size. The bootstrap component contains the full set of bootstrapped coefficients as well as the basic statistics derived from them. Available generic functions are summary(), plot(), coef(), and vcov().

3.5 Filtered parameters

The filtered time-varying parameters of an estimated model can be directly obtained from the output of the `gas()` function. However, to investigate the uncertainty associated with these parameters, the `gas_filter()` function can be used. This function also supports forecasting and provides two methods. The `simulated_coefs` method calculates a path of time-varying parameters for each simulated coefficient set, assuming asymptotic normality with a given variance-covariance matrix. See Blasques et al. (2016b) for more details on this method. The `given_coefs` method computes a path of time-varying parameters for each supplied coefficient set. Suitable sets of coefficients can be obtained, for example, through the use of the `gas_bootstrap()` function.

An estimated GAS model can be supplied as a `gas` object to the `gas_object` argument:

```
gas_filter(gas_object, method = "simulated_coefs", coef_set = NULL,
  rep_gen = 1000L, t_ahead = 0L, x_ahead = NULL, rep_ahead = 1000L,
  quant = c(0.025, 0.975))
```

Alternatively, the individual arguments including the data, model specification, and estimated coefficients with variance-covariance matrix can be provided:

```
gas_filter(method = "simulated_coefs", coef_set = NULL, rep_gen = 1000L,
  t_ahead = 0L, x_ahead = NULL, rep_ahead = 1000L, quant = c(0.025, 0.975), y,
  x = NULL, distr, param = NULL, scaling = "unit", regress = "joint", p = 1L,
  q = 1L, par_static = NULL, par_link = NULL, par_init = NULL,
  coef_fix_value = NULL, coef_fix_other = NULL, coef_fix_special = NULL,
  coef_bound_lower = NULL, coef_bound_upper = NULL, coef_est = NULL,
  coef_vcov = NULL)
```

The `method` argument determines the approach for capturing uncertainty. For the `given_coefs` method, the `coef_set` argument in the form of a numeric matrix of coefficient sets in rows must be provided. For the `simulated_coefs` method, the `rep_gen` argument representing the number of generated coefficient sets must be provided. If forecasting is desired, the number of observations to forecast can be specified using the `t_ahead` argument, values of exogenous variables for the forecasted period can be provided using the `x_ahead` argument, and the number of simulation repetitions in the forecasted sample can be controlled using the `rep_ahead` argument. The desired quantiles can be specified using the `quant` argument.

The function returns a list of S3 class `gas_filter` with three components: `data`, `model`, `filter`. The `data` component contains the supplied time series and exogenous variables. The `model` component contains the specification of the model structure and size. The `filter` component contains in-sample and possibly out-of-sample means, standard deviations, and quantiles of the time-varying parameters and scores. Available generic functions are `summary()` and `plot()`.

3.6 Supplementary functions for distributions

The `distr()` function can be utilized to retrieve a list of distributions and their parametrizations supported by the `gas()` function. To narrow down the output and focus on specific distributions, arguments such as `filter_type` or `filter_dim`, among others, can be specified. The output is in the form of a data.frame with columns providing information on the distributions such as the data type, dimension, orthogonality, and default parameterization.

To work with individual distributions, the **gasmodel** package offers several functions. The `distr_density()` function computes the density of a given distribution, the `distr_mean()` function computes the mean of a given distribution, the `distr_var()` function computes the variance of a given distribution, the `distr_score()` function computes the score of a given distribution, the `distr_fisher()` function computes the Fisher information of a given distribution, and the `distr_random()` function generates random observations from a given distribution. Each of these functions can be supplied with arguments specifying the distribution and the parametrization, namely `distr`, `param`, `par_link`. It is important to note that while the `gas()` function may automatically set the logarithmic/logistic link for time-varying parameters, it must be set manually for the distribution functions. Additionally, a vector of parameter values must be provided to the `f` argument. Some functions may also require an observation to be provided to the `y` argument. For detailed usage instructions, please refer to the documentation for each individual function.

4 Example usage

4.1 Analysis of toilet paper sales

To demonstrate the practical application of the package, we conduct an analysis on the dynamics of toilet paper sales. The dataset `toilet_paper_sales` includes the daily number of toilet paper packs sold in a European store during the years 2001 and 2002, along with a promo variable indicating whether the product was featured in a campaign. In addition to the promo dummy variable, we utilize dummy variables for each day of the week, which are already supplied in the dataset.

There are some missing values, corresponding to the days when the store was closed. It is not necessary to remove these values as the `gas()` function is capable of handling them. When missing observations occur, the time-varying parameters are reinitialized—parameter values are set to their initial value (either the unconditional value or the value supplied by the `par_init` argument), and the score is set to zero. Naturally, missing observations are excluded from the evaluation of the likelihood function.

```
data("toilet_paper_sales")
y <- toilet_paper_sales$quantity
x <- as.matrix(toilet_paper_sales[3:9])
```

Given that our primary variable of interest is in the form of counts, we employ a count distribution. The `distr()` function can be used to obtain a list of appropriate distributions.

```
distr(filter_type = "count", filter_dim = "uni", filter_default = TRUE)$distr
```

```
#> [1] "dpois"    "geom"     "negbin"   "pois"     "zigeom"   "zinegbin" "zipois"
```

We start our analysis by utilizing the Poisson distribution, which is the customary initial choice in count data analysis. The promo and day of the week dummy variables are included as exogenous variables. The dynamics are specified as a regression model with dynamic errors. We estimate the model by the `gas()` function.

```
est_pois <- gas(y = y, x = x, distr = "pois", regress = "sep")
est_pois
```

```
#> GAS Model: Poisson Distribution / Mean Parametrization / Unit Scaling
#>
#> Coefficients:
#>                    Estimate Std. Error   Z-Test Pr(>|Z|)
#> log(mean)_omega   2.6517736  0.0436953  60.6878  < 2e-16 ***
#> log(mean)_beta1  -0.0015755  0.0264302  -0.0596  0.95247
#> log(mean)_beta2  -0.0792304  0.0271167  -2.9218  0.00348 **
#> log(mean)_beta3  -0.0108654  0.0265633  -0.4090  0.68251
#> log(mean)_beta4   0.0554222  0.0257702   2.1506  0.03151 *
#> log(mean)_beta5  -0.8179358  0.0352252 -23.2202  < 2e-16 ***
#> log(mean)_beta6  -1.7635113  0.0538292 -32.7612  < 2e-16 ***
#> log(mean)_beta7   0.7063463  0.0361533  19.5375  < 2e-16 ***
#> log(mean)_alpha1  0.0142384  0.0012708  11.2042  < 2e-16 ***
#> log(mean)_phi1    0.9818186  0.0121071  81.0946  < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Log-Likelihood: -2111.266, AIC: 4242.532, BIC: 4288.128
```

The output of the `gas()` function follows the following convention for naming coefficients. The first part of a coefficient name (before the underscore) refers to the parameter of the distribution and its transformation. The names of the individual distribution parameters are listed in the `distributions` vignette. For nonnegative parameters, the logarithmic transformation, `log()`, can be applied, while for nonnegative parameters less than 1, the logit transformation, `logit()`, can be used. The second part of the coefficient name (after the underscore) refers to either the constant (`omega`), the regression coefficient (`beta`), the score coefficient (`alpha`), or the autoregressive coefficient (`phi`), as described in (3), (4), and (5). For `beta`, the number denotes the corresponding exogenous variable. For `alpha` and `phi`, the number denotes the lag.

The Poisson distribution can be quite restrictive in practice as it necessitates equidispersion, meaning that the mean of the variable is equal to the variance. Subsequently, we opt for a more versatile distribution—the negative binomial distribution, which accommodates overdispersion, allowing the variance to be greater than the mean.

```
est_negbin <- gas(y = y, x = x, distr = "negbin", regress = "sep")
est_negbin
```

```
#> GAS Model: Negative Binomial Distribution / NB2 Parametrization / Unit Scaling
#>
#> Coefficients:
#>                    Estimate Std. Error   Z-Test  Pr(>|Z|)
#> log(mean)_omega   2.6384526  0.0562032  46.9449 < 2.2e-16 ***
#> log(mean)_beta1  -0.0100824  0.0357634  -0.2819   0.77800
#> log(mean)_beta2  -0.0772088  0.0366240  -2.1081   0.03502 *
#> log(mean)_beta3  -0.0155342  0.0361813  -0.4293   0.66767
#> log(mean)_beta4   0.0452483  0.0357004   1.2674   0.20500
#> log(mean)_beta5  -0.8240699  0.0430441 -19.1448 < 2.2e-16 ***
#> log(mean)_beta6  -1.7736613  0.0589493 -30.0879 < 2.2e-16 ***
#> log(mean)_beta7   0.7037864  0.0481655  14.6118 < 2.2e-16 ***
#> log(mean)_alpha1  0.0256734  0.0035329   7.2670 3.676e-13 ***
#> log(mean)_phi1    0.9769718  0.0175857  55.5549 < 2.2e-16 ***
#> dispersion        0.0349699  0.0051488   6.7918 1.107e-11 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Log-Likelihood: -2059.834, AIC: 4141.668, BIC: 4191.824
```

We compare the models based on the Poisson and negative binomial distributions using the Akaike information criterion (AIC).

```
AIC(est_pois, est_negbin)
```

```
#>            df      AIC
#> est_pois   10 4242.532
#> est_negbin 11 4141.668
```

Based on the AIC, the negative binomial model is preferred, as the addition of the overdispersion parameter distinctly improves the fit. Nevertheless, the coefficient estimates, standard errors, and p-values are quite similar in both models. The coefficients for working days are close to zero. Note that Monday is treated as the baseline. In contrast, Saturdays and, even more so, Sundays exhibit a significant drop in sales. Promoting the product in a campaign significantly increases sales. The coefficient estimates themselves need to be interpreted with respect to the utilized logarithmic link function. The score and autoregressive coefficients have the expected positive signs. Furthermore, the autoregressive coefficient is close to one, suggesting high persistence. In Figure 1, we visualize the time-varying mean with the logarithmic transformation using the `plot()` generic function.

```
plot(est_negbin)
```

Next, we project sales for the following year utilizing the `forecast()` function with the default `mean_path` method. This method computes time-varying parameters assuming zero score and subsequently generates the mean of the time series. By manipulating the promo dummy variable, we can perform a what-if analysis. In Figure 2, we illustrate the forecasted sales, considering the promotion of the product throughout the entire year.

```
x_ahead <- cbind(kronecker(matrix(1, 53, 1), diag(7)), 1)[3:367, -1]
fcst_negbin <- gas_forecast(est_negbin, t_ahead = 365, x_ahead = x_ahead)
plot(fcst_negbin)
```

The estimated coefficients are subject to uncertainty. The `gas()` function reports standard errors and p-values based on the empirical Hessian justified by the asymptotic theory. However, if the sample size is small, it may be appropriate to derive the standard errors and p-values using the bootstrap method. The `gas_bootstrap()` function offers both parametric and block bootstrap options. This

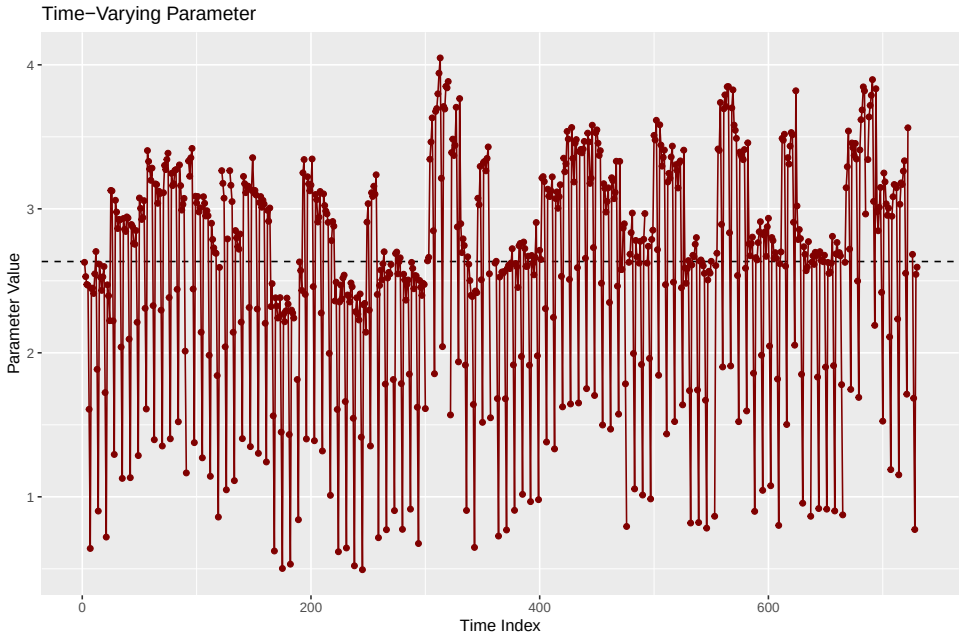

Figure 1: The fitted logarithm of the mean. The horizontal dashed line represents the unconditional value of the logarithm of the mean.

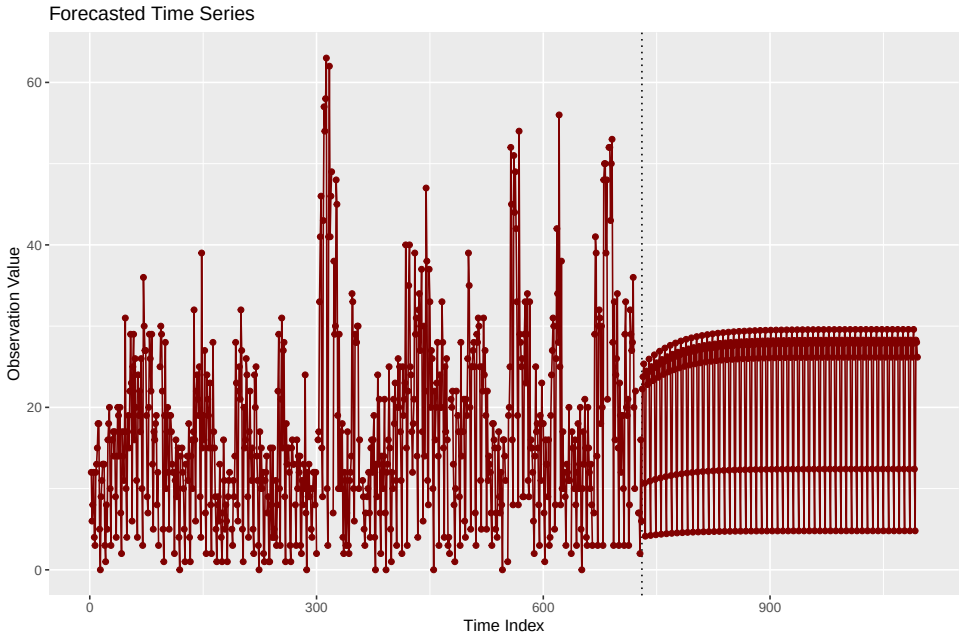

Figure 2: The forecasted toilet paper sales for the next year. The product is assumed to be always promoted. The vertical dotted line divides the in-sample and out-of-sample periods.

process could be computationally intensive, depending on factors such as the number of repetitions, the quantity of observations, the complexity of the model structure, and the optimizer used. To expedite the computation, the function supports parallelization through the arguments `parallel_function` and `parallel_arguments`. As a simple illustration, we demonstrate the parametric bootstrap with only 10 repetitions. To obtain more accurate estimates of the tails and confidence intervals, it is recommended to use a few thousand repetitions.

```
set.seed(42)
boot_negbin <- gas_bootstrap(est_negbin, rep_boot = 10)
```

We have a relatively large sample, and the bootstrapped standard errors closely align with those obtained from the empirical Hessian. Note that standard deviations can also be obtained using the `vcov()` generic function for both `est_negbin` and `boot_negbin`.

```
est_negbin$fit$coef_sd - boot_negbin$bootstrap$coef_sd
```

```
#>  log(mean)_omega  log(mean)_beta1  log(mean)_beta2  log(mean)_beta3
#>    -0.0126390016     0.0062104350    -0.0020216750    -0.0040419542
#>  log(mean)_beta4  log(mean)_beta5  log(mean)_beta6  log(mean)_beta7
#>     0.0040558995     0.0186693207    -0.0017401517     0.0033140886
#> log(mean)_alpha1   log(mean)_phi1       dispersion
#>    -0.0009915866     0.0058394816     0.0008697142
```

As the estimated coefficients are subject to uncertainty, the fitted values of time-varying parameters are also uncertain. To acquire the standard errors and confidence bands of the logarithm of the mean, we employ the `gas_filter()` function with the default `simulated_coefs` method. This method calculates a path of time-varying parameters for each simulated coefficient set under the assumption of asymptotic normality with a given variance-covariance matrix.

```
set.seed(42)
flt_negbin <- gas_filter(est_negbin, rep_gen = 100)
```

In our case, the confidence bands are relatively narrow, averaging 0.16 in width.

```
mean(diff(t(flt_negbin$filter$par_tv_quant[, 1, ])))
```

```
#> [1] 0.1560328
```

4.2 Other case studies

For another example of using the package, we refer to the `case_durations` vignette, which loosely follows the paper by Tomanová and Holý (2021). This case study examines the timing of bookshop orders in the fashion of autoregressive conditional duration (ACD) models. It employs the generalized gamma distribution with a dynamic scale parameter.

The `case_rankings` vignette replicates the case study by Holý and Zouhar (2022). It analyzes the annual results of the Ice Hockey World Championships, which take the form of time-varying rankings. The analysis utilizes both stationary and random walk specifications, along with the Plackett–Luce distribution incorporating dynamic worth parameters.

4.3 Limitations and customization

There are many GAS models with distributions or structures that are not supported by the **gasmodel** package. These include copula models (De Lira Salvatierra and Patton, 2015), matrix models (Opschoor et al., 2018), censoring models (Harvey and Liao, 2023), models with parameter interactions (Holý and Tomanová, 2022), sports rating models (Gorgi et al., 2019), semiparametric models (Blasques et al., 2016a), Markov regime-switching models (Bazzi et al., 2017), and spatio-temporal models (Catania and Billé, 2017). The vignette `customization` further discusses these limitations and demonstrates how the package can be customized to accommodate some of these models.

5 Conclusion

The purpose of the **gasmodel** package is to provide researchers, analysts, and data scientists with a versatile toolkit for a broad spectrum of GAS models in R. While it is important to note that not all GAS models found in the literature are supported by the package due to their diverse nature, the package still provides a solid foundation. For some specific GAS models, modifications of the package may be required, or an alternative specialized package/code may prove to be a better option. Nevertheless, the **gasmodel** package offers considerable flexibility for specifying dynamics, and it boasts an extensive array of probability distribution options.

6 Acknowledgments

The work on this paper was supported by the Czech Science Foundation under project 23-06139S and the personal and professional development support program of the Faculty of Informatics and Statistics, Prague University of Economics and Business.

References

D. Ardia, K. Boudt, and L. Catania. Downside risk evaluation with the R package GAS. *The R Journal*, 10(2):410–421, 2018. doi: 10.32614/rj-2018-064. [p23]

D. Ardia, K. Boudt, and L. Catania. Generalized autoregressive score models in R: The GAS package. *Journal of Statistical Software*, 88(6):1–28, 2019. doi: 10.18637/jss.v088.i06. [p23]

M. Artemova, F. Blasques, J. van Brummelen, and S. J. Koopman. Score-driven models: Methods and applications. *Oxford Research Encyclopedia of Economics and Finance*, 2022. doi: 10.1093/acrefore/9780190625979.013.671. [p27]

M. Bazzi, F. Blasques, S. J. Koopman, and A. Lucas. Time-varying transition probabilities for Markov regime switching models. *Journal of Time Series Analysis*, 38(3):458–478, 2017. doi: 10.1111/jtsa.12211. [p35]

F. Blasques, S. J. Koopman, and A. Lucas. Stationarity and ergodicity of univariate generalized autoregressive score processes. *Electronic Journal of Statistics*, 8(1):1088–1112, 2014. doi: 10.1214/14-ejs924. [p27]

F. Blasques, S. J. Koopman, and A. Lucas. Information-theoretic optimality of observation-driven time series models for continuous responses. *Biometrika*, 102(2):325–343, 2015. doi: 10.1093/biomet/asu076. [p27]

F. Blasques, J. Ji, and A. Lucas. Semiparametric score driven volatility models. *Computational Statistics & Data Analysis*, 100(2013):58–69, 2016a. doi: 10.1016/j.csda.2015.04.003. [p35]

F. Blasques, S. J. Koopman, K. Łasak, and A. Lucas. In-sample confidence bands and out-of-sample forecast bands for time-varying parameters in observation-driven models. *International Journal of Forecasting*, 32(3):875–887, 2016b. doi: 10.1016/j.ijforecast.2015.11.018. [p29, 31]

F. Blasques, P. Gorgi, S. J. Koopman, and O. Wintenberger. Feasible invertibility conditions and maximum likelihood estimation for observation-driven models. *Electronic Journal of Statistics*, 12(1):1019–1052, 2018. doi: 10.1214/18-ejs1416. [p27]

F. Blasques, A. Lucas, and A. C. van Vlodrop. Finite sample optimality of score-driven volatility models: Some Monte Carlo evidence. *Econometrics and Statistics*, 19:47–57, 2021. doi: 10.1016/j.ecosta.2020.03.010. [p27]

F. Blasques, J. van Brummelen, S. J. Koopman, and A. Lucas. Maximum likelihood estimation for score-driven models. *Journal of Econometrics*, 227(2):325–346, 2022. doi: 10.1016/j.jeconom.2021.06.003. [p27]

F. Blasques, V. Holý, and P. Tomanová. Zero-inflated autoregressive conditional duration model for discrete trade durations with excessive zeros. *Studies in Nonlinear Dynamics and Econometrics*, 28(5):673–702, 2024. doi: 10.1515/snde-2022-0008. [p27]

S. Blazsek and A. Licht. Dynamic conditional score models: A review of their applications. *Applied Economics*, 52(11):1181–1199, 2020. doi: 10.1080/00036846.2019.1659498. [p27]

G. Bodin, R. Saavedra, C. Fernandes, and A. Street. ScoreDrivenModels.jl: A Julia package for generalized autoregressive score models. 2020. [p23]

T. Bollerslev. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3): 307–327, 1986. doi: 10.1016/0304-4076(86)90063-1. [p27]

H. W. Borchers. *Package 'pracma'*, 2023. URL https://cran.r-project.org/package=pracma. [p29]

F. Calvori, F. Cipollini, and G. M. Gallo. Go with the flow: A GAS model for predicting intra-daily volume shares. 2013. [p27]

L. Catania and A. G. Billé. Dynamic spatial autoregressive models with autoregressive and heteroskedastic disturbances. *Journal of Applied Econometrics*, 32(6):1178–1196, 2017. doi: 10.1002/jae.2565. [p35]

D. Creal, S. J. Koopman, and A. Lucas. A general framework for observation driven time-varying parameter models. 2008. URL https://www.tinbergen.nl/discussion-paper/2649. [p24]

D. Creal, S. J. Koopman, and A. Lucas. A dynamic multivariate heavy-tailed model for time-varying volatilities and correlations. *Journal of Business & Economic Statistics*, 29(4):552–563, 2011. doi: 10.1198/jbes.2011.10070. [p27]

D. Creal, S. J. Koopman, and A. Lucas. Generalized autoregressive score models with applications. *Journal of Applied Econometrics*, 28(5):777–795, 2013. doi: 10.1002/jae.1279. [p23, 24, 25]

R. A. Davis, W. T. M. Dunsmuir, and S. B. Street. Observation-driven models for Poisson counts. *Biometrika*, 90(4):777–790, 2003. doi: 10.1093/biomet/90.4.777. [p27]

I. De Lira Salvatierra and A. J. Patton. Dynamic copula models and high frequency data. *Journal of Empirical Finance*, 30:120–135, 2015. doi: 10.1016/j.jempfin.2014.11.008. [p27, 35]

R. F. Engle and J. R. Russell. Autoregressive conditional duration: A new model for irregularly spaced transaction data. *Econometrica*, 66(5):1127–1162, 1998. doi: 10.2307/2999632. [p27]

R. V. Fonseca and F. Cribari-Neto. Bimodal Birnbaum-Saunders generalized autoregressive score model. *Journal of Applied Statistics*, 45(14):2585–2606, 2018. doi: 10.1080/02664763.2018.1428734. [p27]

P. Gilbert and R. Varadhan. *Package 'numDeriv'*, 2022. URL https://cran.r-project.org/package=numDeriv. [p29]

P. Gorgi, S. J. Koopman, and R. Lit. The analysis and forecasting of tennis matches by using a high dimensional dynamic model. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 182 (4):1393–1409, 2019. doi: 10.1111/rssa.12464. [p27, 35]

P. R. Hansen, P. Janus, and S. J. Koopman. Realized Wishart-GARCH: A score-driven multi-asset volatility model. 2016. [p27]

A. Harvey, S. Hurn, D. Palumbo, and S. Thiele. Modelling circular time series. *Journal of Econometrics*, 239(1):105450, 2024. doi: 10.1016/j.jeconom.2023.02.016. [p25, 27]

A. C. Harvey. *Dynamic Models for Volatility and Heavy Tails: With Applications to Financial and Economic Time Series*. Cambridge University Press, New York, 2013. doi: 10.1017/cbo9781139540933. [p23, 24]

A. C. Harvey. Score-driven time series models. *Annual Review of Statistics and Its Application*, 9(1): 321–342, 2022. doi: 10.1146/annurev-statistics-040120-021023. [p27]

A. C. Harvey and T. Chakravarty. Beta-t-(E)GARCH. 2008. [p24, 27]

A. C. Harvey and R. Ito. Modeling time series when some observations are zero. *Journal of Econometrics*, 214(1):33–45, 2020. doi: 10.1016/j.jeconom.2019.05.003. [p27]

A. C. Harvey and Y. Liao. Dynamic tobit models. *Econometrics and Statistics*, 26:72–83, 2023. doi: 10.1016/j.ecosta.2021.08.012. [p27, 35]

V. Holý. Analyzing and forecasting success in the Men's Ice Hockey World (Junior) Championships using a dynamic ranking model. *Journal of Quantitative Analysis in Sports*, 21(3):211–235, 2025. doi: 10.1515/jqas-2024-0137. [p26]

V. Holý and P. Tomanová. Modeling price clustering in high-frequency prices. *Quantitative Finance*, 22 (9):1649–1663, 2022. doi: 10.1080/14697688.2022.2050285. [p27, 35]

V. Holý and J. Zouhar. Modelling time-varying rankings with autoregressive and score-driven dynamics. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 71(5):1427–1450, 2022. doi: 10.1111/rssc.12584. [p27, 35]

R. J. Hyndman. The ARIMAX model muddle, 2010. URL https://robjhyndman.com/hyndsight/arimax/. [p26]

S. J. Koopman and R. Lit. Forecasting football match results in national league competitions using score-driven time series models. *International Journal of Forecasting*, 35(2):797–809, 2019. doi: 10.1016/j.ijforecast.2018.10.011. [p27]

S. J. Koopman, A. Lucas, and M. Scharth. Predicting time-varying parameters with parameter-driven and observation-driven models. *Review of Economics and Statistics*, 98(1):97–110, 2016. doi: 10.1162/rest_a_00533. [p27]

S. J. Koopman, R. Lit, A. Lucas, and A. Opschoor. Dynamic discrete copula models for high-frequency stock price changes. *Journal of Applied Econometrics*, 33(7):966–985, 2018. doi: 10.1002/jae.2645. [p27]

R. Lit, S. J. Koopman, and A. C. Harvey. *Time Series Lab - Score Edition*, 2021. URL https://timeserieslab.com. [p23]

A. Opschoor, P. Janus, A. Lucas, and D. van Dijk. New HEAVY models for fat-tailed realized covariances and returns. *Journal of Business & Economic Statistics*, 36(4):643–657, 2018. doi: 10.1080/07350015.2016.1245622. [p35]

G. Sucarrat. betategarch: Simulation, estimation and forecasting of Beta-Skew-t-EGARCH models. *The R Journal*, 5(2):137–147, 2013. doi: 10.32614/rj-2013-034. [p23]

R. Taylor. *PyFlux: An Open Source Time Series Library for Python*, 2018. URL https://github.com/rjt1990/pyflux. [p23]

P. Tomanová and V. Holý. Clustering of arrivals in queueing systems: Autoregressive conditional duration approach. *Central European Journal of Operations Research*, 29(3):859–874, 2021. doi: 10.1007/s10100-021-00744-7. [p35]

J. Ypma and S. G. Johnson. *Package 'nloptr'*, 2020. URL https://cran.r-project.org/package=nloptr. [p29]

Vladimír Holý
Prague University of Economics and Business
Department of Econometrics
Winston Churchill Square 1938/4, 130 67 Prague 3, Czechia
ORCiD: 0000-0003-0416-0434
vladimir.holy@vse.cz